

COSC 3P32 – Introduction to Database Systems

Winter 2026

Assignment #1 Sample Solutions

Question 1 [40 marks]

See ER diagram in the separate file 3p32assn1SolutionER.pdf.

This diagram does capture all of the mentioned constraints.

Question 2 [30 marks]

My set of relations is as follows, with some brief explanations.

Actor(stagename, realname, abirthyear, abirthplace, unum)

Director(dname, dbirthyear, dbirthplace)

VideoD(vtitle, year, studio, genre, dname, dpay)

- *This combines Video and Directs, because Directs is a 1-many relationship with total participation (there is exactly 1 director for each video).*
- *Note also that because Movie and Show do not cover Video (i.e. there can be some videos that are neither movies or shows), we need to have all of these as separate tables.*

Movie(vtitle, year, mlength, rating)

Show(vtitle, year, seasons)

ActsIn(stagename, vtitle, year, apay, role)

EpisodeOf(etitle, enum, eseaon, elength, vtitle, year)

- *This combines Episode and BelongsTo, since Episode is a weak entity set and BelongsTo is its identifying relationship.*

Account(aid, aphone)

ViewerShares(vname, vpref, aid)

- *This combines Viewer and Shares, since Viewer is a weak entity set and Shares is its identifying relationship.*

Interval(istart, iend)

Watches(vname, aid, vtitle, year, istart, iend)

SQL statements:

For all of the following CREATE TABLE statements, the actual types used for the attributes are unimportant at this stage.

```
CREATE TABLE Actor (
    stagename CHAR(30),
    realname CHAR(30),
    abirthyear INTEGER,
    abirthplace CHAR(30),
    unum INTEGER,
    PRIMARY KEY (stagename),
    UNIQUE (unum))
```

Note: unum is unique because no two actors can have the same unum. But it can't be the primary key because some actors might not have unum = null.

```
CREATE TABLE VideoD (
    vtitle      CHAR(30),
    year        INTEGER,
    studio      CHAR(30),
    genre       CHAR(30),
    dname       CHAR(30) NOT NULL,
    dpay        INTEGER,
    PRIMARY KEY (vtitle, year),
    FOREIGN KEY (dname) REFERENCES Director
        ON DELETE NO ACTION
        ON UPDATE CASCADE)
```

```
CREATE TABLE Movie (
    vtitle      CHAR(30),
    year        INTEGER,
    mlength     INTEGER,
    rating      INTEGER,
    PRIMARY KEY (vtitle, year),
    FOREIGN KEY (vtitle, year) REFERENCES VideoD
        ON DELETE CASCADE
        ON UPDATE CASCADE)
```

```
CREATE TABLE Show (
    vtitle      CHAR(30),
    year        INTEGER,
    seasons     INTEGER,
    PRIMARY KEY (vtitle, year),
    FOREIGN KEY (vtitle, year) REFERENCES VideoD
        ON DELETE CASCADE
        ON UPDATE CASCADE)
```

```
CREATE TABLE ActsIn (
    stagename    CHAR(30),
    vtitle       CHAR(30),
    year         INTEGER,
    apay         INTEGER,
    role         CHAR(30),
    PRIMARY KEY (stagename, vtitle, year),
    FOREIGN KEY (stagename) REFERENCES Actor
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    FOREIGN KEY (vtitle, year) REFERENCES VideoD
        ON DELETE CASCADE
        ON UPDATE CASCADE)
```

```
CREATE TABLE EpisodeOf (
    etitle      CHAR(30),
    enum        INTEGER,
    eseaon      INTEGER,
    elength     INTEGER,
    vtitle      CHAR(30) NOT NULL,
    year        INTEGER NOT NULL,
    PRIMARY KEY (etitle, vtitle, year),
    UNIQUE (enum, eseaon, vtitle, year),
```

```

FOREIGN KEY (vtitle, year) REFERENCES Show
ON DELETE CASCADE,
ON UPDATE CASCADE)

```

Notes: (enum, eseaason, vtitle, year) must be unique because it is specified that no two episodes of the same show (as specified by vtitle and year) can have the same combination of enum and eseaason. Also, it should definitely be “cascade” for both “on delete” and “on update” because when a show is deleted/updated all of its episodes should also be deleted/updated. Finally, I have specified that both vtitle and year are not null, to ensure total participation.

```

CREATE TABLE Account (  aid          INTEGER,
                        aphone        CHAR(10),
                        PRIMARY KEY (aid),
                        UNIQUE (aphone))

```

Note: aphone should be unique because it is specified that a phone number can only be associated with one account.

```

CREATE TABLE ViewerShares (  vname        CHAR(30),
                             vpref        CHAR(30),
                             aid          INTEGER NOT NULL,
                             PRIMARY KEY (vname, aid),
                             FOREIGN KEY aid REFERENCES Account
                             ON DELETE CASCADE
                             ON UPDATE CASCADE)

```

Notes: It should definitely be “cascade” for both “on delete” and “on update” because when an account is deleted/updated all of the viewers sharing that account should also be deleted/updated. Finally, aid is not null to ensure total participation.

```

CREATE TABLE Interval (  istart        TIMESTAMP,
                         iend          TIMESTAMP,
                         PRIMARY KEY (istart, iend))

```

```

CREATE TABLE Watches (  vname        CHAR(30),
                         aid          INTEGER,
                         vtitle       CHAR(30),
                         year         INTEGER,
                         istart        TIMESTAMP,
                         iend          TIMESTAMP,
                         PRIMARY KEY (vname, aid, vtitle, year, istart, iend),
                         FOREIGN KEY (vname, aid) REFERENCES ViewerShares
                         ON DELETE CASCADE
                         ON UPDATE CASCADE,
                         FOREIGN KEY (vtitle, year) REFERENCES Video
                         ON DELETE CASCADE
                         ON UPDATE CASCADE,
                         FOREIGN KEY (istart, iend) REFERENCES Interval
                         ON DELETE NO ACTION
                         ON UPDATE NO ACTION)

```

Notes: If a viewer is deleted/updated then information on what they watch should also be deleted/updated. Similarly, if a video is deleted/updated then information on who watched it should also be deleted/updated.

For the project, it is fine to incorporate Interval into Watches.

Brief summary of constraints captured:

- *All primary key constraints*
- *All foreign key constraints*
- *1-many relationships: these are all captured by combining the tables (this includes the two weak entities) and/or by setting the primary keys appropriately*
- *Total participation: unless this also coincides with a 1-many relationship, these constraints are **not** currently captured. This includes:*
 - o *Every actor must act in at least one video*
 - o *Every director must direct at least one video*
 - o *Every account has at least one viewer*
 - o *Every show has at least one episode*
- *Overlap and covering constraints:*
 - o *As Movie and Show do not cover Video, we keep them as 3 separate tables, with Movie and Show both referencing Video. This ensures that movie information is stored in both Movie and Video, and Show information is stored in both Show and Video, while any other type of video is stored only in Video, not in Movie or Show.*
 - o *Movie and Show should not overlap, but this is not currently captured in the tables.*

Question 3 [21 marks]

Consider the relations you defined in Question 2 above. For all of the following SQL statements, you are to adhere as closely as possible to the SQL format used in the course notes (that also corresponds to that used in the textbook).

- a) [6 marks] Write SQL statement(s) to add the following movie and associated information: “Highlander II: The Quickening”, 1991, from “Fox” studio, “science fiction” genre, with length 91 minutes and rating 0, and directed by Russell Mulcahy who was paid \$1 million. (It really is a terrible movie. A completely unnecessary and nonsensical sequel.)

```
INSERT
INTO VideoD(vtitle, year, studio, genre, dname, dpay)
VALUES ('Highlander II: The Quickening', 1991, 'Fox', 'Science Fiction',
'Russell Mulcahy', 1000000)
```

```
INSERT
INTO Movie(vtitle, year, mlength, rating)
VALUES ('Highlander II: The Quickening', 1991, 91, 0)
```

- b) [3 marks] Write SQL statement(s) to set all episodes of “The Simpsons” to have a length of 22 minutes.

```
UPDATE   EpisodeOf E
SET      E.elength = 22
WHERE    E.vtitle = 'The Simpsons'
```

- c) [6 marks] Write SQL statement(s) to update the stage name of “Michael Fox” to “Michael J. Fox”. Given the referential integrity constraints chosen in your answer to Question 2 above, clearly explain what happens when this statement is executed, and why.

```
UPDATE Actor A
SET A.stagename = 'Michael J. Fox'
WHERE A.stagename = 'Michael Fox'
```

Explanation:

One other table has a foreign key that references stagename: ActsIn, with CASCADE for “on update”. As a result, when the stagename is modified in Actor it will also be modified in ActsIn.

- d) [6 marks] Write SQL statement(s) to delete the account with account ID 1234567. Given the referential integrity constraints chosen in your answer to Question 2 above, clearly explain what happens when this statement is executed, and why.

```
DELETE
FROM Account A
WHERE A.aid = 1234567
```

Explanation:

One other table has a foreign key that references stagename: ViewerShares, with CASCADE for “on delete”. As a result, when the account is deleted then all viewers for that account will also be deleted.

Question 4 [9 marks]

Given the relations you defined in Question 2 above, express each of the following queries in SQL. Again, you must adhere as closely as possible to the SQL format used in the course notes (that also corresponds to that used in the textbook).

- a) Find the titles of all movies with rating of at least 60.

```
SELECT M.vtitle
FROM Movie M
WHERE M.rating >= 60
```

- b) Find the stage names of actors with real name “Michael Douglas”.

```
SELECT A.stagename
FROM Actor A
WHERE A.realname = 'Michael Douglas'
```

- c) Find the episode titles of all episodes with episode number 1 and episode season 1.

```
SELECT E.etable
FROM EpisodeOf E
WHERE E.enum = 1 AND E.esession = 1
```